Exploring Spiking Neural Networks for Real-Time Information Processing

Technical report

Karol Chlasta[1] and Grzegorz Marcin Wójcik[12]

[1] Department of Computer Science
Polish-Japanese Academy of Information Technology
{kchlasta,gmwojcik}@pja.edu.pl
[2] Department of Neuroinformatics and Biomedical Engineering
Maria Curie-Skłodowska University in Lublin, Poland
gmwojcik@live.umcs.edu.pl

1 Project idea

Our brain is built of about 10^{11} interacting neural cells that form a large and complex neural network. Different authors estimate a broad range of 75–125 billion neurons in the whole brain [7]. In biological systems, the neurons are organised into neural microcircuits to be able to process complex input information. The function of each microcircuit or column is dependent on neuron types and its location in the brain [4]. The spiking neural networks built of Hodgkin-Huxley neurons behave like Liquid State Machines[6].

By default the processing of spiking neural networks (SNN) should be accelerated with multi core central processing units (CPU), rather than graphics processing units (GPU). This is because of CPU's larger flexibility of task and thread level parallelism. SNN simulations could be less efficient on GPUs, as the high GPU efficiency for traditional artificial neural networks (ANN) relies on fixed arrays [1].

1.1 Liquid State Machines

Alan Turing showed that one can construct machines that are universal for digital sequential offline computing [13]. In contrast to this common computational model, Maass's Liquid State Machine model introduced in 2002 [9] does not require information to be stored in stable states of a computational system. In the same year a Liquid Computer term was coined as a novel strategy for real-time computing on time series [11]. Such a computer consists of the four main components:

- Input $i(\cdot)$, a continuous input stream.
- Liquid column, or a filter L^N that maps the input into function $Y^N(t)$:

$$Y^N(t) = (L^N i)(t). \tag{1}$$

- Memory-less readout map m^N.
- Output $o(t)$:

$$o(t) = m^N(Y^N(t)). \tag{2}$$

In Maass's LSM architecture described in [9] the function of time series $i(\cdot)$ is injected as input into the liquid column L^N, creating at time t the liquid state $Y^N(t)$, which is transformed by a memory-less readout map m^N to generate an output $o(t)$.

Such approach to computations is much more suitable for parallel real-time computing on analog input streams. It generated several new models of so called reservoir computing, which are also often called liquid state machine, or LSM [11]. In general, these type of models are created of two parts: the reservoir of a highly recurrent spiking neural network, and a readout function characterised by a simple learning function.

The input always fed into the *reservoir*, often known as *liquid*, which acts as a filter. Then the state of the liquid, called the *state vector*, is used as input for the *readout function*. As a result, the readout function trains on the output of the liquid, with no training happening within the reservoir itself.

This process has been analogised with dropping objects into a container of liquid and subsequently reading the ripples created to classify the objects—this analogy popularised the term Liquid State Machine [8, 12].

Maass introduced two macroscopic properties of LSM, *a separation property* (SP) and *an approximation property* (AP), that can provide universal computational power regardless of specific structure or implementation. SP measures the dispersion between projected liquid states from different classes, whereas the AP indicates the concentration of the liquid states that belong to the same class.

As defined by Mass [9], the universal computational power is provided when the two abstract properties are met: the class of basis filters from which the liquid filters L^N are composed satisfies the point-wise separation property and the class of functions from which the readout maps n^N are drawn satisfies the approximation property.

We define that a class CB of filters has the *point-wise separation property* with regard to input functions from U^n if for any two functions $u(\cdot),\ v(\cdot) \in \mathrm{U}^n$ with $u(s) \neq v(s)$ for some $s \leq 0$ there exists filter $B \in CB$ that separates $u(\cdot)$ and $v(\cdot)$, that is, $(Bu)(0) \neq (Bv)(0)$ for any two functions $u(\cdot)$, $v(\cdot) \in \mathrm{U}^n$ with $u(s) \neq v(s)$ for some $s \leq 0$. Simple examples for the classes CB of filters that have this property are the class of all:

- delay filters $u(\cdot) \mapsto \mathrm{u}^{t_0}(\cdot)$ for $t_0 \in R$
- linear filters, with impulse responses of the form $h(t) = e^{-at}$ with $a > 0$
- non-linear filters, as defined in Neural Systems as Nonlinear Filters [10]

A liquid filter L^N of an LSM N is said to be *composed* of filters from CB if there are finitely many filters $B_1, \ldots, B_m$ in CB, to which we refer as basis filters in this context, so that $(L^N u)(t) = (B_1 u)(t), \ldots, (B_m u)(t))$ for all $t \in R$ and all input functions $u(\cdot)$ in U^n. In other words, the output of L^N for a particular input u is simply the vector of outputs given by these finitely many basis filters for this input u.

A class CF of functions has the *approximation property* if for any $m \in N$, any compact (e.g., closed and bounded) set $X \subseteq R_m$, any continuous function $h : X \to R$, and any given $\rho > 0$, there exists some f in CF so that $|h(x) - f(x)| \leq \rho$, for all x ∈ X. The definition for the case of functions with multidimensional output is analogous.

To summarise, the theorems proposed by Maas [9] imply that there are no serious a priori limits for the computational power of LSMs on a continuous functions of time, and they provide a theoretical foundation for approximating any biologically relevant computation on discrete spike trains in continuous time by LSMs.

In this approach only the synapses of the readout neurons need to be adapted for a particular computational task, and the current state $x(t)$ of a microcircuit at time t holds all information about preceding inputs.

The architecture is under active development. Recently [14] proposed to use a group of locally connected LSM reservoirs to form an ensemble of liquids. This approach could simulate higher levels of connectivity in LSMs that are in close proximity, and lower levels of connectivity for these that are spatially further apart.

1.2 Objectives

The main objective is to build and examine a large, modular artificial neural network of spiking neurons in a simulation of a LSM system processing visual signals (from 50k to 1M of neurons) on a large computational cluster. The secondary goal is to explore the properties of LSM.

2 Project Activities

2.1 Modelling Visual System

We propose a modular, bio-inspired model of a visual system that consists of two main components, an *Input* (acting as a retina of the system) and *Liquid* (acting as a visual cortex, built of a single LSM column). Our new Hodgkin-Huxley Liquid State Machine (HHLSM) model uses high fidelity multi-compartmental neurons with voltage-activated sodium and potassium channels. We build on a well known conductance-based model describing how action potentials in neurons are initiated and propagated in electrical circuit [16]. The simulation parameters we used can be organised into four groups:

- Main resistances R_x = 0.3 Ω, R_n = 0.33 Ω.

- Capacitance C_n = 0.01 F and potential incl. E_n = 0.07 V, E_k = 0.0594 V (soma compartment), E_k = 0.07 V (dendrite).
- Conductance G_K = 360 Ω^{-1} and G_{Na} = 1200 Ω^{-1} (for each of the ionic channels).
- Physical characteristics, a soma with circular shape and diameter of 30 μ, dendrites and axon length of 100 μ.

The exact values for these parameters were achieved experimentally by [16] to provide a high biological fidelity of the model. A detailed description of the Hodgkin-Huxley model can be found in [5].

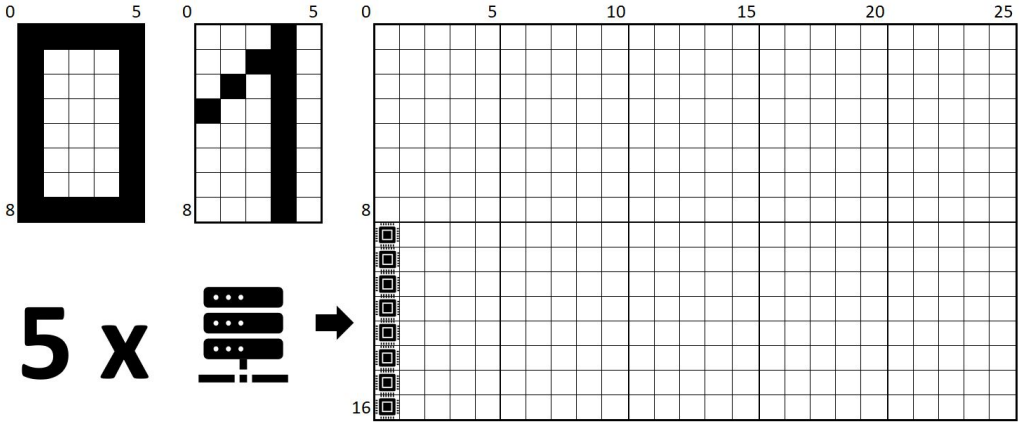

Figure 1: Retina of the proposed visual system with the stimulating patterns (Input).

This model is different to prior models e.g. the one presented in by Wójcik [15] in 2006. The current model called RetNet(5x8, 1) has a single LSM column. It is intended to become a building block of a much larger, and more complex model that could be built of 1M+ neurons, using a novel simulation setup. The final Retina of that target model was built of smaller RetNet(5x8, 1) models on a 125×16 rectangle-shaped grid and divided into 5 patches (25×16), with each patch connected to 10 HHLSM columns forming Lateral Geniculate Nuclei (LGN) and the ensemble of cortical microcircuit. The retinal cells are only connected to the LSM column through the retina's top LGN layer. Each HHLSM consists of 1024 neural cells placed in a rectangular cuboid of 8×5×25 (1000 HH neural cells). The structure of each column in the models is the same. The target model therefore contains 50 columns forming the Liquids stimulated by Retina(s).

There are 90% of excitatory connections established among layers and neurons of each layer and 10% of inhibitory connections. For simplicity, we did not implement the pathway of the possible inter-column connections. Each connection in the model is characterised with a delay parameter and random weight. We can treat

the Liquid as a single hyper-column made of 60 independent neural columns, a part of periodic structure of the simulated cortex.

2.2 Future SOC Lab Resources

We have used two types of resources from Future SOC Lab. The first type was a powerful Ubuntu 18.04 bionic virtual machine that we used as our development workstation (vm-20211005-001.fsoc.hpi.uni-potsdam.de, running Linux 4.15.0-143-generic on x86_64 architecture). We received a privileged access to the virtual machine allowing us to install the software we needed to compile the latest version of GENESIS simulation environment [2] (version 2.4 from May 2019). The machine was equipped with 8 CPUs: 8 x 2.00 GHz, memory (RAM): 7.79 GB, local disk: we used 9.843 of 75.522 GB as of November 5th, 2021. The machine was provisioned from a 1000-Core cluster consisting of 25 Nodes, each 40 CPU cores, 1 TiB RAM (Quanta QSSC-S4R Server System).

The other resource we received access to was an instance of SAP HANA database. [3]. We have managed to connect to the database using Python, but due to limited amounts of data generated, and lack of time we decided not to load any data, and focus our efforts on setting up docker containers with GENESIS to be able to build a more advanced simulation setup.

2.3 Limitations

With the HPI Future SOC Lab's support the key limitation related to lack of computational resources was resolved. The assigned VM was not always available due to an on-going data centre upgrade, so authors decided to rebuild a smaller scale development environment on their Raspberry Pi 4B computer.

3 First Results

We built and ran the initial building blocks for the large 1M+ LSM model. We tested two small RetNet(5x8,1) blocks consisting of 2 LSM columns and 2080 spiking neurons in total. We show that our pipeline can execute the two blocks of the simulation in parallel. This is achieved without using designated CPU steering commands that are provided by GENESIS ("@1, @2").

A baseline model RetNet(28x28,4) took about 8 mins 18 seconds to execute on Raspberry Pi 4B. Simulation of a single second of the RetNet(5x8,1) block takes on average 1 min and 29 sec with HPI's VM, and 2 mins 3 seconds on Raspberry Pi 4B.

HPI's Future SOC VM running on the HP 1000 core cluster is on average only 37.79% faster than our 4 core Raspberry Pi 4B, but we expect it to be much more scalable in the next stage of the project in 2022.

Table 1: Summary of SNN simulations (1 second of LSM system, 20000 steps). Execution in CPU seconds, memory and data in MBytes.

Computer	Model	Pattern	Elements	Execution	Memory	Data
HPI's VM	RetNet(5x8, 1)	0	28124	90.24 s	3.48 MB	1.43 MB
HPI's VM	RetNet(5x8, 1)	1	28113	87.92 s	3.48 MB	1.28 MB
Own RPi 4B	RetNet(5x8, 1)	0	28124	123.32 s	3.48 (677.94) MB	1.38 MB
Own RPi 4B	RetNet(5x8, 1)	1	28113	122.18 s	3.48 (649.72) MB	1.19 MB
Own RPi 4B	RetNet(28x28, 4)	0	133010	474.68 s	16.42 (549.70) MB	5.80 MB
Own RPi 4B	RetNet(28x28, 4)	1	132987	474.68 s	16.42 (499.99) MB	3.73 MB

4 Next Steps

A simple, modular HH spiking neural network model RetNet(5x8,1) was designed, built and simulated. The main objective was to build a large model, and to measure its separation property. We have partially achieved the key goals. Until November 2021 we only simulated 2k HH neurons organised in a two part rectangle-shaped grid (retina) and 2 cortical microcircuits (1 node, 8 CPU cores) using HPI resources.

In the upcoming months we would like to continue our research to run larger models (e.g. 1M+ model), and focus on incorporating Slurm and/or Docker into our pipeline. We hope that HPI Future SOC Lab will allow us to experiment with Slurm Workload Manager and building multi-architecture Docker containers for GENESIS.

In spite of the small amounts of data gathered so far, we would still like to experiment with SAP HANA. The intention is to add it into the pipeline to evaluate the readout signal using HANA ML pipelines. We imagine that the target simulation setup could cover even 100000 cortical microcircuits running on (ideally) all 25 nodes and 1000 cores.

5 Acknowledgements

We thank Michael Connolly for proofreading, and Bernhard Rabe, Tobias Pape, Ayleen Oswald for their on-going support at HPI Future SOC Lab throughout the year 2021.

6 Appendix

Listing 1: A code snippet presenting GENESIS functions written to support the modelling of visual system described in this report.

```
function make_circuit_2d_output(net, nx, ny, filename)
 int i,j
 create spikehistory {net}-history
 setfield ^ filename {filename}.spike append 0 ident_toggle 1
 for (i=1; i<={nx}; i={i+1})
  for (j=1; j<={ny}; j={j+1})

              addmsg {net}_{i}_{j}/soma/spike {net}-history SPIKESAVE

        end
    end

echo {net} spike activity saved to file {filename}.spike
end

function make_circuit_3d(protocell, net, nx, ny, nz)
str protocell
int i,j,k

 for (i=1; i<={nx};i={i+1})
  for (j=1; j<={ny};j={j+1})
    for (k=1; k<={nz};k={k+1})

      copy {protocell} {net}_{i}_{j}_{k}

      position {net}_{i}_{j}_{k} { {array_minx} + ({sep_x} * {i}) } \
                                 { {array_miny} + ({sep_y} * {j}) } \
                                 { {array_minz} + ({sep_z} * {k}) }
    end
  end
 end
end
```

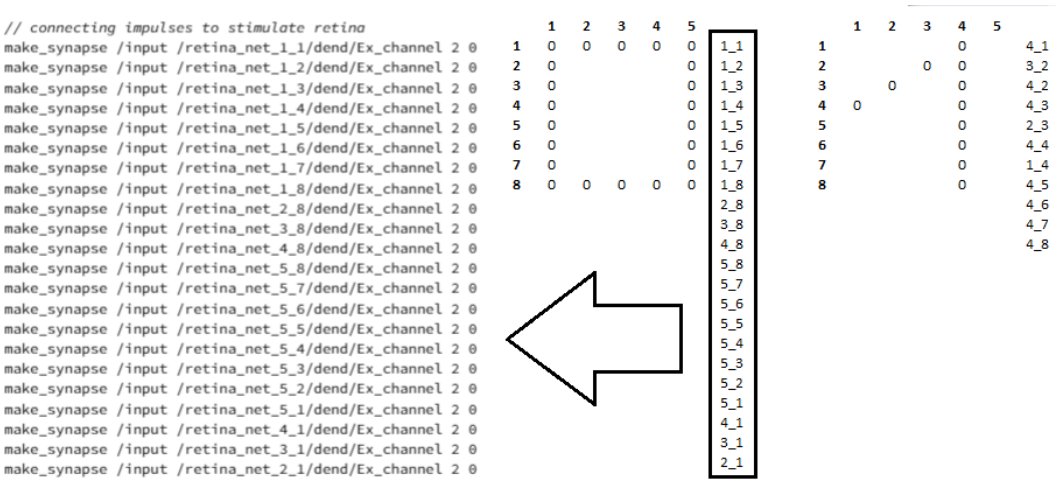

Figure 2: A GENESIS code snippet presenting the implementation of the pattern "o" stimulating retina's 5x8 grid.

References

[1] M. A. Bhuiyan, V. K. Pallipuram, M. C. Smith, T. Taha, and R. Jalasutram. "Acceleration of spiking neural networks in emerging multi-core and GPU architectures". In: *2010 IEEE International Symposium on Parallel & Distributed Processing, Workshops and Phd Forum (IPDPSW)*. IEEE. 2010, pages 1–8.

[2] J. M. Bower, D. Beeman, and M. Hucka. "The GENESIS simulation system". In: (2003).

[3] F. Färber, N. May, W. Lehner, P. Große, I. Müller, H. Rauhe, and J. Dees. "The SAP HANA Database–An Architecture Overview." In: *IEEE Data Eng. Bull.* 35.1 (2012), pages 28–33.

[4] A. Gupta, Y. Wang, and H. Markram. "Organizing principles for a diversity of GABAergic interneurons and synapses in the neocortex". In: *Science* 287.5451 (2000), pages 273–278.

[5] A. L. Hodgkin and A. F. Huxley. "A quantitative description of membrane current and its application to conduction and excitation in nerve". In: *The Journal of physiology* 117.4 (1952), pages 500–544.

[6] W. A. Kamiński and G. M. Wójcik. "Liquid state machine built of Hodgkin-Huxley neurons-pattern recognition and informational entropy". In: *Annales Universitatis Mariae Curie-Sklodowska, sectio AI–Informatica* 1.1 (2015), pages 1–7.

[7] R. Lent, F. A. Azevedo, C. H. Andrade-Moraes, and A. V. Pinto. "How many neurons do you have? Some dogmas of quantitative neuroscience under revision". In: *European Journal of Neuroscience* 35.1 (2012), pages 1–9.

[8] W. Maass. “Liquid state machines: motivation, theory, and applications”. In: *Computability in context: computation and logic in the real world* (2011), pages 275–296.

[9] W. Maass, T. Natschläger, and H. Markram. “Real-time computing without stable states: A new framework for neural computation based on perturbations”. In: *Neural computation* 14.11 (2002), pages 2531–2560.

[10] W. Maass and E. D. Sontag. “Neural systems as nonlinear filters”. In: *Neural computation* 12.8 (2000), pages 1743–1772.

[11] T. Natschläger, W. Maass, and H. Markram. “The "liquid computer": A novel strategy for real-time computing on time series”. In: *Special issue on Foundations of Information Processing of TELEMATIK* 8.ARTICLE (2002), pages 39–43.

[12] D. Norton and D. Ventura. “Improving the separability of a reservoir facilitates learning transfer”. In: *2009 International Joint Conference on Neural Networks*. IEEE. 2009, pages 2288–2293.

[13] C. E. Shannon. “A universal Turing machine with two internal states”. In: *Automata studies* 34 (1956), pages 157–165.

[14] P. Wijesinghe, G. Srinivasan, P. Panda, and K. Roy. “Analysis of liquid ensembles for enhancing the performance and accuracy of liquid state machines”. In: *Frontiers in neuroscience* 13 (2019), page 504.

[15] G. M. Wojcik and W. A. Kaminski. “Liquid computations and large simulations of the mammalian visual cortex”. In: *International Conference on Computational Science*. Springer. 2006, pages 94–101.

[16] T. Yorozu, M. Hirano, K. Oka, and Y. Tagawa. “Electron spectroscopy studies on magneto-optical media and plastic substrate interface”. In: *IEEE translation journal on magnetics in Japan* 2.8 (1987), pages 740–741.